Technical Report: How the Closed-Form Update for $\mathbf{M}$ Is Embedded in the Current Prop Skew-t Backend

April 25, 2026

1 Purpose of This Report

This report is written as a *verification memo*, not as a proposal note. Its goal is to answer two narrow technical questions for the current prop implementation in `D:/Github/spatial-skew-t`:

1. Is the current implementation of the closed-form maximum likelihood estimator of the low-rank covariance matrix $\mathbf{M}$ algebraically consistent with the Tzeng–Huang theorem after the basis construction used by this backend?
2. How is that closed-form covariance update embedded inside the current skew-t sampler without replacing the rest of the Morris-style hierarchical structure?

The report is therefore designed to clarify what is exact, what is only a working/profile step, and what evidence would be needed to claim that the current code writes the intended estimator correctly.

2 Files That Matter

The relevant code paths are:

- `code/R/prop/prop_basis.R`: basis construction, trimming, and orthonormalization.
- `code/R/prop/prop_covariance.R`: closed-form update for the low-rank block and the construction of $\mathbf{G}$ and $\mathbf{R}$.
- `code/R/prop/prop_modules.R`: high-level wrappers such as `update_M_closed_form(...)` and `build_R_from_G(...)`.
- `code/R/prop/mcmc_prop.R`: the hybrid skew-t sampler that calls the covariance update.
- `code/analysis/simstudy_prop/run-prop.R`: runner that passes `prop_k` and `prop_cov_update_every` into the backend and saves the resulting fit objects.

The *runner* governs how jobs are launched and saved. The actual closed-form update for $\mathbf{M}$ lives in the *backend*. The scientific correctness question therefore belongs primarily to the backend, with the runner only responsible for exposing the correct control parameters and recording metadata.

3 Implemented Prop Skew-t Model

3.1 Observation-Level Form

For replicate $t = 1, \ldots, T$, let $\mathbf{Y}_t \in \mathbb{R}^n$ denote the observed response vector at the n observed spatial sites. The current prop backend can be written as

$$\mathbf{Y}_t = \mathbf{X}_t \boldsymbol{\beta} + \lambda \mathbf{z}_{g,t} + \mathbf{D}_t \mathbf{v}_t, \tag{1}$$

where:

- $\mathbf{X}_t$ is the covariate matrix at replicate t;
- $\boldsymbol{\beta}$ is the regression coefficient vector;
- λ is the skewness coefficient;
- $\mathbf{z}_{g,t}$ is the site-level skew latent vector induced by the current partition/knots structure;
- $\mathbf{D}_t = \mathrm{diag}\{\tau_{g,t}(s_1)^{-1/2}, \ldots, \tau_{g,t}(s_n)^{-1/2}\}$ is the scale matrix built from the current local precisions;
- $\mathbf{v}_t$ is the latent Gaussian spatial effect with correlation matrix $\mathbf{R}$.

Equation (1) matches the current code structure in `mcmc_prop.R`: the mean part is formed as `x.beta + lambda * zg`, while the scale is carried by `taug` through the standardization step.

3.2 What Is Preserved From the Morris Hierarchy

The current prop implementation does *not* replace the whole Morris-style hierarchy. It keeps the main non-Gaussian latent blocks:

- $\boldsymbol{\beta}$ update;
- λ update when skewness is enabled;
- local scale update through τ and the site-level map `taug`;
- latent skew process update through `z` and `zg`;
- knot/partition updates when `nknots > 1`.

What changes is only the latent spatial dependence block $\mathbf{v}_t$. The Matérn-driven covariance update is replaced by a low-rank spatial random effects block that is refreshed from a closed-form estimator of $\mathbf{M}$ and σ_ξ^2.

4 Conditional Gaussian Working Residuals

4.1 Standardized Residuals Used by the Covariance Update

Given the current state

$$(\boldsymbol{\beta}, \lambda, \mathbf{z}_{g,t}, \tau_{g,t}), \qquad t = 1, \ldots, T,$$

define the standardized residual vector

$$\mathbf{r}_t = \mathbf{D}_t^{-1}\big(\mathbf{Y}_t - \mathbf{X}_t\boldsymbol{\beta} - \lambda \mathbf{z}_{g,t}\big), \qquad t = 1,\ldots,T, \tag{2}$$

where $\mathbf{D}_t^{-1} = \operatorname{diag}\{\tau_{g,t}(s_1)^{1/2},\ldots,\tau_{g,t}(s_n)^{1/2}\}$.

This matches the code exactly:

$$\texttt{residuals\_std} = \sqrt{\texttt{taug}} \odot \big(\texttt{y} - \texttt{x\_beta} - \lambda \texttt{zg}\big),$$

implemented in `update_residuals(...)`.

4.2 Why Theorem 2 Is Applied to $\mathbf{r}_t$, Not Directly to $\mathbf{Y}_t$

Tzeng–Huang's theorem is derived for Gaussian spatial random effects replicates with a common covariance model. In the current backend, the original observations are not Gaussian because of the skew-t hierarchy. The theorem is therefore applied to the *conditionally Gaussian working residuals* $\mathbf{r}_t$, not to $\mathbf{Y}_t$ directly.

This is the first crucial interpretive point:

> the current backend does not claim that the full skew-t model admits a closed form MLE for $\mathbf{M}$; it claims that, conditional on the current latent blocks, the Gaussian spatial dependence layer can be refreshed by a theorem-based closed-form update.

That is why the method should be described as a *hybrid Bayesian–profile* algorithm rather than a full Bayesian sampler for $\mathbf{M}$.

5 Basis Construction and Why the Theorem Simplifies

5.1 Fixed Basis and Rank Reduction

The current backend fixes the basis matrix through `build_prop_basis(...)`. Let the requested basis size be K_{req}, but let the numerically retained rank after trimming and orthonormalization be r. The code returns

$$\mathbf{Q} \in \mathbb{R}^{n\times r}, \qquad \mathbf{Q}'\mathbf{Q} = \mathbf{I}_r.$$

This orthonormalization step matters because it changes how Theorem 2 should be read in the implementation. The theorem is written for a general basis matrix $\mathbf{F}$, but in the current backend the working basis has already been orthonormalized. Therefore the factor

$$(\mathbf{F}'\mathbf{F})^{-1/2}$$

collapses to $\mathbf{I}_r$ after basis preprocessing.

5.2 The Working Empirical Covariance

Collect the residual replicates as

$$\mathbf{R} = \begin{bmatrix}\mathbf{r}_1 & \cdots & \mathbf{r}_T\end{bmatrix}, \qquad \mathbf{S} = \frac{1}{T}\mathbf{R}\mathbf{R}'.$$

Then with the orthonormalized basis $\mathbf{Q}$, the theorem's matrix

$$\mathbf{H} = (\mathbf{F}'\mathbf{F})^{-1/2}\mathbf{F}'\mathbf{S}\mathbf{F}(\mathbf{F}'\mathbf{F})^{-1/2}$$

reduces to

$$\mathbf{H} = \mathbf{Q}'\mathbf{S}\mathbf{Q}. \tag{3}$$

The code computes exactly this quantity by

$$\texttt{projected} = \mathbf{Q}'\mathbf{R}, \qquad \texttt{h\_mat} = \frac{1}{T}\texttt{projected}\,\texttt{projected}'.$$

Hence

$$\texttt{h\_mat} = \mathbf{Q}'\mathbf{S}\mathbf{Q}.$$

This is the second crucial point of the report:

> the implementation is not omitting the $(\mathbf{F}'\mathbf{F})^{-1/2}$ factor by mistake; it has already absorbed that factor by orthonormalizing the basis before the closed-form update is called.

6 Theorem 2 Update and Its Code Mapping

6.1 The Working Theorem

Given the orthonormalized basis $\mathbf{Q}$, let

$$\mathbf{H} = \mathbf{Q}'\mathbf{S}\mathbf{Q} = \mathbf{P}\,\mathrm{diag}(d_1, \ldots, d_r)\mathbf{P}'$$

be its eigendecomposition, where $d_1 \geq \cdots \geq d_r \geq 0$. Let $d_0 = 0$, and let

$$L^* = \max\left\{ L \in \{1, \ldots, r\} : d_L > \max\left(\frac{1}{n-L}\Big(\mathrm{tr}(\mathbf{S}) - \sum_{k=1}^{L} d_k\Big), \sigma_\epsilon^2 \right) \right\},$$

with $L^* = 0$ if the set is empty.

Then the theorem-based update is

$$\widehat{\sigma}_\xi^2 = \max\left\{ \frac{1}{n-L^*}\left(\mathrm{tr}(\mathbf{S}) - \sum_{k=1}^{L^*} d_k\right) - \sigma_\epsilon^2,\ 0 \right\}, \tag{4}$$

and

$$\widehat{\mathbf{M}} = \mathbf{P}\,\mathrm{diag}(\widehat{d}_1, \ldots, \widehat{d}_r)\mathbf{P}', \qquad \widehat{d}_k = \max(d_k - \widehat{\sigma}_\xi^2 - \sigma_\epsilon^2, 0). \tag{5}$$

In the current backend, σ_ϵ^2 is passed as `prop_sigma_eps_sq` and defaults to 0.

6.2 Implementation-Level Regularization

The code introduces one additional regularization step beyond the theorem:

$$\widehat{\sigma}_\xi^2 \leftarrow \max\{\widehat{\sigma}_\xi^2, \texttt{sigma\_floor}\}.$$

This does *not* change the structure of the theorem-based update, but it does mean the implementation is a *numerically regularized* version of the closed-form estimator rather than the exact raw formula with no lower bound.

6.3 Theorem-to-Code Audit Table

Mathematical object	Backend object	Function	Comment
$\mathbf{Q}$	`F_obs`	`build_basis_matrix(...)`	Orthonormalized observation basis
$\mathbf{Q}_*$	`F_pred`	`build_basis_matrix(...)`	Prediction basis rows
$\mathbf{r}_t$	`residuals_std[, t]`	`update_residuals(...)`	Standardized working residual replicate
$\mathbf{S}$	implicit	`prop_closed_form_update(...)`	Never formed explicitly as $n \times n$
$\mathbf{H}$	`h_mat`	`prop_closed_form_update(...)`	Computed as $Q'SQ$
$\mathbf{P}$	`rotation`	`prop_closed_form_update(...)`	Eigenvectors of $\mathbf{H}$
$(d_1, \ldots, d_r)$	`projected_eigs`	`prop_closed_form_update(...)`	Eigenvalues of $\mathbf{H}$
L^*	`lstar`	`prop_select_lstar(...)`	The theorem's retained index
$\widehat{\sigma}_\xi^2$	`sigma_xi_sq`	`prop_closed_form_update(...)`	Nugget/fine-scale variance update
$(\widehat{d}_1, \ldots, \widehat{d}_r)$	`lowrank_eigs`	`prop_closed_form_update(...)`	Shrunk eigenvalues
$\widehat{\mathbf{M}}$	`M_hat`	`update_M_closed_form(...)`	Reconstructed as $P\,\mathrm{diag}(\widehat{d})P'$

7 From $\widehat{\mathbf{M}}$ to the Working Correlation Matrix

7.1 Rotated-Basis Representation

Although `update_M_closed_form(...)` reconstructs $\widehat{\mathbf{M}}$ explicitly, the subsequent covariance construction uses the equivalent rotated-basis form. Let

$$\mathbf{B} = \mathbf{QP}, \qquad \boldsymbol{\Lambda} = \mathrm{diag}(\widehat{d}_1, \ldots, \widehat{d}_r).$$

Then

$$\mathbf{Q}\widehat{\mathbf{M}}\mathbf{Q}' = \mathbf{QP}\boldsymbol{\Lambda}\mathbf{P}'\mathbf{Q}' = \mathbf{B}\boldsymbol{\Lambda}\mathbf{B}'. \tag{6}$$

The current code uses $\mathbf{B}$ and $\boldsymbol{\Lambda}$ rather than multiplying by $\widehat{\mathbf{M}}$ directly. This is mathematically equivalent, and it is the reason why `build_R_from_G(...)` only requires `rotation`, `lowrank_eigs`, and `sigma_xi_sq`.

7.2 From G to R

The working covariance matrix is

$$\mathbf{G} = \mathbf{B}\boldsymbol{\Lambda}\mathbf{B}' + \widehat{\sigma}_\xi^2\mathbf{I}_n. \tag{7}$$

The current backend then defines

$$\mathbf{A} = \mathrm{diag}(\mathbf{G}), \qquad \mathbf{R} = \mathbf{A}^{-1/2}\mathbf{G}\mathbf{A}^{-1/2}.$$

This is implemented in `prop_make_cov_state(...)`.

This standardization step is scientifically important:

- the theorem gives an update for $\mathbf{G}$, a covariance matrix;
- the skew-t sampler is run using $\mathbf{R}$, a correlation matrix;
- the scale of the observations remains controlled by τ, not by $\mathbf{G}$.

Therefore the current backend does *not* optimize the full skew-t likelihood over a covariance parameterization. Instead it performs a theorem-based refresh of $\mathbf{G}$ and then converts that covariance block to a correlation matrix before re-entering the skew-t hierarchy.

8 How the Update Is Embedded Inside the Sampler

8.1 Backend-Level Flow

The runner `run-prop.R` passes two key controls into `mcmc(...)`:

- `prop_k`: requested basis size;
- `prop_cov_update_every`: how often the closed-form covariance refresh is performed.

Inside `mcmc_prop.R`, the relevant steps are:

1. Build the fixed basis $\mathbf{Q}$ and $\mathbf{Q}_*$.
2. Initialize the latent skew and scale blocks.
3. Compute the current working residual block $\{\mathbf{r}_t\}_{t=1}^T$.
4. Update $(\widehat{\mathbf{M}}, \widehat{\sigma}_\xi^2)$ from those residuals.
5. Build $\mathbf{R}$, its inverse, log-determinant pieces, and the prediction state.
6. Continue the skew-t updates for $\boldsymbol{\beta}$, λ, τ, $\mathbf{z}$, and knots.
7. Every `prop_cov_update_every` iterations, refresh the covariance block again from the new residuals.

8.2 Iteration-Level Pseudo-Code

Pseudo-code for one MCMC iteration

1. Given current $(\boldsymbol{\beta}, \lambda, \tau, \mathbf{z}, \text{knots})$, form $\mathbf{r}_t$ by (2).
2. If thresholding or missingness is present, impute latent y conditional on the current covariance state.
3. Update $\boldsymbol{\beta}$ and λ.
4. Update knots/partition state if knots are free.
5. Update τ.
6. Update $\mathbf{z}$ if skewness is enabled.

7. Recompute $\mathbf{r}_t$.
8. If the iteration index is a covariance-refresh step, compute $(\widehat{\mathbf{M}}, \widehat{\sigma}_\xi^2)$, then rebuild $\mathbf{R}$.
9. Save the current draw, including prediction draws generated from the updated covariance state.

This pseudo-code is the correct interpretation of the current backend: the closed-form estimator is embedded as a *refresh step within a larger skew-t sampler*, not as a stand-alone Gaussian fitting procedure.

9 What Is Exact, and What Is Approximate

9.1 Exact Statements

The following claims are exact for the current code:

1. The basis is orthonormalized before the theorem-based update is called.
2. The matrix fed into the eigendecomposition is exactly $Q'SQ$ for the working residual matrix.
3. The reconstructed matrix $\widehat{\mathbf{M}} = \mathbf{P}\,\mathrm{diag}(\widehat{d})\mathbf{P}'$ is the theorem's closed-form estimator in the orthonormalized basis.
4. The covariance state is built from the rotated-basis representation $\mathbf{B}\boldsymbol{\Lambda}\mathbf{B}' + \widehat{\sigma}_\xi^2\mathbf{I}$, which is algebraically equivalent to $Q\widehat{M}Q' + \widehat{\sigma}_\xi^2\mathbf{I}$.

9.2 Approximate or Profile Statements

The following claims should *not* be overstated:

1. The theorem is not being applied to the full skew-t data model; it is being applied to conditionally standardized residuals.
2. The standardization $\mathbf{G} \mapsto \mathbf{R}$ is an additional modeling choice made to preserve the scale/dependence separation.
3. The sampler does not draw $\mathbf{M}$ from a posterior distribution. It plugs in a theorem-based update inside the MCMC loop.
4. The lower bound `sigma_floor` is an implementation regularizer and is not part of the raw closed-form theorem.

Hence the right label is:

> *hybrid Bayesian–profile skew-t inference with a theorem-based closed-form covariance refresh.*

10 Why This Still Preserves the Skew-t Process Structure

The key scientific concern is whether introducing the closed-form covariance refresh destroys the integrity of the skew-t process. The answer is no, for the following reason:

- the non-Gaussian features of the model still come from the latent skew process, the local scale process, and the threshold/missing-data machinery;
- the covariance refresh only replaces the Gaussian spatial dependence layer that sits underneath those blocks;
- prediction still uses the same skew and scale ingredients through `lambda`, `z_pred`, and `sigma_pred`.

So the present backend is not "Gaussianized" overall. It remains a skew-t process model with a low-rank Gaussian dependence component that is refreshed by a closed-form estimator.

11 What Must Be Verified to Claim the Estimator Is Correct

To move from "the code matches the intended algebra" to "the estimator is implemented correctly," the following checks should be documented.

11.1 Algebraic Invariants

For each covariance refresh:

1. Verify $Q'Q = I_r$ numerically.
2. Verify `rotation` is orthonormal.
3. Verify all `lowrank_eigs` are nonnegative.
4. Verify $\widehat{\mathbf{M}}$ is positive semidefinite.
5. Verify `effective_rank` $\leq \min(r, T)$.
6. Verify $\widehat{\sigma}_\xi^2 \geq$ `sigma_floor` > 0.

11.2 Small-n, Small-T Numerical Check

Construct a small Gaussian test case where:

- Q is fixed and orthonormal;
- σ_ϵ^2 is known;
- the residual replicates are truly Gaussian.

Then compare:

1. the code's $(\widehat{\mathbf{M}}, \widehat{\sigma}_\xi^2)$;
2. a direct implementation of the theorem using matrix algebra outside the backend;

3. a brute-force likelihood optimization over $\mathbf{M}$ and σ_ξ^2 in the same reduced-rank Gaussian model.

If those three agree up to numerical tolerance, then the core MLE update is very likely coded correctly.

11.3 Sampler-Integration Check

Independently of the theorem itself, one should also verify:

1. when `prop_cov_update_every = 1`, the covariance block refreshes at every iteration;
2. when `prop_cov_update_every > 1`, the covariance state stays fixed between refreshes;
3. the saved `fit.1$prop` metadata is consistent with the basis rank, $\widehat{\sigma}_\xi^2$, and the requested k.

12 Bottom-Line Conclusion

From a mathematical and code-audit perspective, the current prop backend is best described as follows:

1. The theorem-based update for $\mathbf{M}$ is implemented on the orthonormalized basis, so the code correctly uses $Q'SQ$ rather than the full $(F'F)^{-1/2}F'SF(F'F)^{-1/2}$ expression.
2. The code reconstructs $\widehat{\mathbf{M}}$ in the theorem's eigenbasis and then uses the equivalent rotated-basis form to build the covariance matrix.
3. The backend does not claim an exact skew-t MLE for $\mathbf{M}$; it performs a closed-form Gaussian covariance refresh inside a larger skew-t sampler.
4. Therefore the current implementation is scientifically coherent as a hybrid Bayesian–profile algorithm, provided the remaining numerical checks are passed.

The most important practical implication is this:

> if the estimator is questioned, the first place to look is not the sampler logic but the theorem-to-code mapping for the orthonormalized basis and the $\mathbf{G} \mapsto \mathbf{R}$ standardization step.

A Numerical Verification Appendix

This appendix turns the previous discussion into a concrete verification protocol. Its purpose is not to demonstrate predictive performance, but to determine whether the current implementation of the closed-form update for $\mathbf{M}$ is numerically consistent with the intended theorem and whether that update is embedded correctly in the current skew-t backend.

The verification logic should proceed in three layers:

1. verify the theorem-based update in a pure Gaussian reduced-rank setting;
2. verify the covariance-to-correlation conversion and prediction-state construction;
3. verify that the closed-form step is refreshed at the intended points of the skew-t sampler.

The order matters. If the first layer fails, then discrepancies observed in the full skew-t sampler cannot be interpreted, because the covariance update itself has not yet been certified.

A.1 Appendix A: Deterministic Theorem Replication Test

A.1.1 Goal

The first check should isolate the theorem from the rest of the model. The question is:

> if we feed the backend an orthonormal basis and truly Gaussian replicates, does the code return the same $(\widehat{\mathbf{M}}, \widehat{\sigma}^2_\xi)$ as a direct matrix-algebra implementation of the theorem?

A.1.2 Recommended Setup

Use a deliberately small configuration, for example

$$n = 12, \qquad r = 4, \qquad T = 40.$$

Construct:

1. an orthonormal matrix $\mathbf{Q} \in \mathbb{R}^{n \times r}$, obtained for example from a QR decomposition of a random Gaussian matrix;
2. a positive semidefinite target covariance
$$\mathbf{M}_{\text{true}} = \mathbf{P}_0 \operatorname{diag}(m_1, \ldots, m_r)\mathbf{P}_0',$$
with $m_k \geq 0$;
3. a positive nugget/fine-scale variance $\sigma^2_{\xi,\text{true}} > 0$;
4. the Gaussian covariance
$$\mathbf{G}_{\text{true}} = \mathbf{Q}\mathbf{M}_{\text{true}}\mathbf{Q}' + \sigma^2_{\xi,\text{true}}\mathbf{I}_n.$$

Then simulate
$$\mathbf{r}_t \overset{\text{iid}}{\sim} N_n(\mathbf{0}, \mathbf{G}_{\text{true}}), \qquad t = 1, \ldots, T.$$

A.1.3 Three Parallel Calculations

For the same simulated residual matrix $\mathbf{R}$, compute $(\widehat{\mathbf{M}}, \widehat{\sigma}^2_\xi)$ in three ways.

Path 1: Backend path Use the current code:

1. call `update_M_closed_form(F_obs = Q, residuals_std = R)`;
2. extract `rotation`, `lowrank_eigs`, `sigma_xi_sq`, and `M_hat`.

Path 2: Direct theorem path Implement the theorem outside the backend:

1. compute
$$\mathbf{S} = \frac{1}{T}\mathbf{R}\mathbf{R}', \qquad \mathbf{H} = \mathbf{Q}'\mathbf{S}\mathbf{Q};$$
2. eigendecompose $\mathbf{H} = \mathbf{P} \operatorname{diag}(d_1, \ldots, d_r)\mathbf{P}'$;
3. compute L^*, $\widehat{\sigma}^2_\xi$, and $\widehat{d}_k = \max(d_k - \widehat{\sigma}^2_\xi, 0)$;
4. reconstruct
$$\widehat{\mathbf{M}}_{\text{dir}} = \mathbf{P} \operatorname{diag}(\widehat{d}_1, \ldots, \widehat{d}_r)\mathbf{P}'.$$

Path 3: Rebuilt rotated-basis path Using the backend output only, reconstruct

$$\widehat{\mathbf{G}}_{\text{rot}} = (\mathbf{Q}\,\texttt{rotation})\ \text{diag}(\texttt{lowrank\_eigs})\,(\mathbf{Q}\,\texttt{rotation})' + \texttt{sigma\_xi\_sq}\,\mathbf{I}_n.$$

This path checks whether the code's rotated-basis representation is numerically equivalent to $Q\widehat{M}Q' + \widehat{\sigma}_\xi^2 I$.

A.1.4 Acceptance Criteria

For a successful replication check, require:

1. $$\max_{i,j} \left| \widehat{M}_{\text{backend},ij} - \widehat{M}_{\text{dir},ij} \right| < 10^{-8};$$

2. $$\left| \widehat{\sigma}^2_{\xi,\text{backend}} - \widehat{\sigma}^2_{\xi,\text{dir}} \right| < 10^{-10};$$

3. $$\max_{i,j} \left| \widehat{G}_{\text{rot},ij} - \left(\mathbf{Q}\widehat{\mathbf{M}}_{\text{backend}}\mathbf{Q}' + \widehat{\sigma}^2_{\xi,\text{backend}}\mathbf{I}_n \right)_{ij} \right| < 10^{-8}.$$

If these fail, then the implementation of the theorem itself should be treated as unresolved, and no higher-level skew-t conclusions should be drawn.

A.2 Appendix B: Direct Likelihood Benchmark

A.2.1 Goal

The deterministic theorem replication test only shows that the code matches the matrix algebra. A second question is whether that matrix algebra actually agrees with the Gaussian likelihood in the reduced-rank model. This check is:

> does the theorem-based update match the optimizer of the Gaussian likelihood in the same low-rank covariance family?

A.2.2 Likelihood to Optimize

For fixed residual replicates $\mathbf{r}_1, \ldots, \mathbf{r}_T$, define

$$\ell(\mathbf{M}, \sigma_\xi^2) = -\frac{T}{2}\log|\mathbf{G}| - \frac{1}{2}\sum_{t=1}^{T} \mathbf{r}_t'\mathbf{G}^{-1}\mathbf{r}_t, \qquad \mathbf{G} = \mathbf{QMQ}' + \sigma_\xi^2\mathbf{I}_n.$$

To keep this benchmark feasible, use only a very small rank such as $r = 2$ or $r = 3$, and optimize over a Cholesky parameterization

$$\mathbf{M} = \mathbf{LL}', \qquad \sigma_\xi^2 = \exp(\eta),$$

with unconstrained parameters in $\mathbf{L}$ and η.

A.2.3 Verification Procedure

1. Use the same Gaussian dataset as Appendix A.
2. Compute the theorem-based estimator $(\widehat{\mathbf{M}}_{\text{thm}}, \widehat{\sigma}^2_{\xi,\text{thm}})$.
3. Independently optimize $\ell(\mathbf{M}, \sigma^2_\xi)$ numerically using `optim(...)` or an equivalent optimizer.
4. Compare the final log-likelihood values and the resulting covariance matrices.

A.2.4 Acceptance Criteria

For the Gaussian reduced-rank benchmark:

1. the theorem-based and optimized log-likelihoods should agree to within roughly 10^{-6} to 10^{-4}, depending on the optimizer tolerance;
2. the resulting covariance matrices should satisfy

$$\max_{i,j} |\widehat{G}_{\text{thm},ij} - \widehat{G}_{\text{opt},ij}|$$

   at a correspondingly small tolerance;
3. if likelihood values disagree materially, inspect whether the difference comes from the σ^2_ξ floor or from optimizer failure.

This benchmark is the strongest practical confirmation that the coded closed-form update is not only algebraically consistent, but also likelihood consistent in the intended Gaussian subproblem.

A.3 Appendix C: $\mathbf{G} \mapsto \mathbf{R}$ Standardization Check

A.3.1 Goal

Even if $\widehat{\mathbf{M}}$ is correct, the current backend uses $\mathbf{R}$, not $\mathbf{G}$, in the skew-t updates. Therefore the standardization step must be verified separately.

A.3.2 Checks to Run

Given a covariance state built by `build_R_from_G(...)`:

1. verify symmetry:
$$\max_{i,j} |R_{ij} - R_{ji}| < 10^{-10};$$
2. verify unit diagonal:
$$\max_{i} |R_{ii} - 1| < 10^{-10};$$
3. verify positive definiteness numerically, for example by checking that the Cholesky factorization of $\mathbf{R}$ succeeds;
4. verify that the stored `prec_obs` is numerically the inverse of $\mathbf{R}$:
$$\max_{i,j} |(\texttt{prec\_obs}\,\mathbf{R} - \mathbf{I})_{ij}| < 10^{-8};$$
5. verify that the cross-covariance and conditional prediction covariance objects are internally consistent with the Schur complement formula.

A.3.3 Why This Appendix Matters

This step is where the theorem leaves its original Gaussian covariance setting and enters the current skew-t backend. Therefore, if any numerical mismatch appears here, the theorem may still be correct while the $\mathbf{G} \mapsto \mathbf{R}$ integration is not.

A.4 Appendix D: Sampler-Integration Verification

A.4.1 Goal

The final verification layer is not about the theorem itself, but about whether the theorem-based refresh is inserted into the MCMC loop at the intended positions.

A.4.2 Recommended Minimal Experiments

Use the smallest practical runs that still exercise the covariance refresh:

1. **Gaussian no-threshold test:** set `method = "gaussian"`, `skew = FALSE`, no censoring, `prop_cov_update_every = 1`.
2. **Skew-*t* integration test:** set `method = "t"`, `skew = TRUE`, with the current latent blocks enabled, and again use `prop_cov_update_every = 1`.
3. **Refresh-frequency test:** repeat the same run with `prop_cov_update_every = 3`.

A.4.3 What to Verify

Refresh timing Check that:

- with `prop_cov_update_every = 1`, the saved `sigma_xi` and `effective_rank` traces can change at every iteration after burn-in;
- with `prop_cov_update_every = 3`, the covariance state remains unchanged between refresh points.

Saved metadata Check that the saved `fit.1$prop` object contains:

- `requested_k`;
- `basis_kept_cols`;
- `basis_rank_kept`;
- `sigma_xi`;
- `effective_rank`;
- `cov_update_every`;
- `workflow = "prop-simstudy"`.

Prediction path Check that prediction draws are produced from the refreshed covariance state and that disabling random prediction noise (`draw = FALSE` in a local test) returns the conditional mean implied by the stored state.

A.5 Appendix E: Suggested Decision Rules

For practical workflow use, the numerical checks above can be summarized into three decision levels.

Green light Proceed with scientific experiments if:

1. Appendix A passes exactly to numerical tolerance;
2. Appendix B shows no meaningful likelihood gap;
3. Appendix C confirms that $\mathbf{R}$, $\mathbf{R}^{-1}$, and the prediction state are internally consistent;
4. Appendix D confirms that the refresh schedule and metadata are correct.

Amber light Proceed only with caution if:

1. the theorem replication passes, but the likelihood benchmark shows small discrepancies traceable to σ_ξ^2 flooring or optimizer tolerance;
2. the sampler integration works, but some metadata fields are incomplete.

Red light Do not interpret prop-vs-Morris experiments if:

1. the theorem replication fails;
2. the rebuilt rotated-basis covariance does not match $Q\widehat{M}Q' + \widehat{\sigma}_\xi^2 I$;
3. the covariance refresh occurs at the wrong iterations;
4. the $\mathbf{G} \mapsto \mathbf{R}$ state is numerically inconsistent.

A.6 Appendix F: Recommended Implementation Order

To avoid conflating errors, the numerical verification should be implemented in the following order:

1. write a stand-alone Gaussian theorem-check script that touches only `prop_basis.R`, `prop_covariance.R`, and `prop_modules.R`;
2. add a direct optimization benchmark for the same tiny Gaussian examples;
3. add a covariance-state validation script for $\mathbf{G}$, $\mathbf{R}$, and the prediction Schur complement;
4. only then run miniature `mcmc_prop.R` smoke tests;
5. only after all of the above pass, interpret large-scale simstudy results.

This staged order matches the statistical logic of the method: verify the closed-form covariance block first, then verify the correlation conversion, then verify the skew-t integration.

References

[1] ShengLi Tzeng and Hsin-Cheng Huang. Resolution Adaptive Fixed Rank Kriging. *Technometrics*, 60(2):198–208, 2018.

[2] Noel Cressie and Gardar Johannesson. Fixed rank kriging for very large spatial data sets. *Journal of the Royal Statistical Society: Series B*, 70(1):209–226, 2008.